

Universidad Nacional de Ingeniería

Facultad de Ingeniería Eléctrica y Electrónica



“Análisis y Comparación de Algoritmos de Caminos Mínimos en Grafos”

Ciclo académico: 2026 – 1

Especialidad: Ingeniería electrónica

Código del curso: BMA09

Nombre del curso: ALGORITMOS Y ESTRUCTURA DE DATOS

Catedrático:

Estudiante:

Simeon Rafael Carlos Kennedy

Camarena Quiquia Andy Jonas

Villanueva Arana Rodrigo Alonso

- **INDICE:**
- **1.Introducción.....**
- **2.Conceptos básicos de grafos.....**
- **3.Problema del camino mínimo.....**
- **4.Relajación de aristas**
- **5.Algoritmo de Dijkstra.....**
- **6.Algoritmo Bellman-Ford.....**
- **7.Algoritmo Floyd-Warshall.....**
- **8.Comparación de los algoritmos.....**
- **9.Selección del algoritmo adecuado**
- **10.Ejemplo práctico comparativo.....**
- **11.Aplicaciones reales.....**
- **12.Implementación computacional.....**
- **13.Complejidad computacional.....**
- **14.Conclusiones.....**
- **15.Recomendaciones.....**

- **16. Referencias bibliográficas.....**

1. INTRODUCCIÓN

Como señalan Cormen, Leiserson, Rivest y Stein en *Introduction to Algorithms*, los algoritmos permiten establecer procedimientos ordenados para resolver diferentes problemas computacionales. Dentro de este campo, los grafos constituyen una herramienta importante para representar conexiones y relaciones entre diferentes elementos. Su aplicación se encuentra en situaciones como redes de comunicación, sistemas de transporte, mapas, rutas y otros problemas relacionados con la búsqueda y optimización de recorridos.

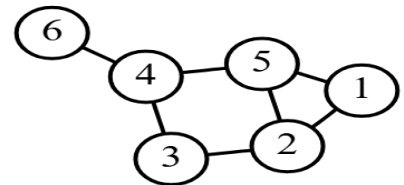
El estudio de los grafos comprende diversos conceptos fundamentales, como los vértices, las aristas, los pesos, los tipos de grafos y las formas de representarlos. Estos elementos permiten modelar diferentes situaciones y analizar las conexiones existentes entre los puntos que forman una red. A partir de estos conceptos se puede abordar uno de los problemas más importantes de la teoría de grafos: el problema del camino mínimo, que busca determinar la ruta de menor costo entre dos vértices o entre diferentes pares de vértices.

Para resolver este problema se utilizan diferentes técnicas y algoritmos. Entre los principales se encuentran Dijkstra, Bellman-Ford y Floyd-Warshall, los cuales presentan diferentes características y condiciones de aplicación. También es fundamental comprender conceptos como la relajación de aristas, los pesos positivos y negativos, los ciclos negativos y la representación de las distancias dentro de un grafo.

En el presente informe se desarrollan los principales conceptos relacionados con los grafos y el problema del camino mínimo, para posteriormente estudiar el funcionamiento de los algoritmos de Dijkstra, Bellman-Ford y Floyd-Warshall. Asimismo, se analizan sus características, procedimientos, ventajas y limitaciones, y se realiza una comparación entre ellos. Finalmente, se explica cómo seleccionar el algoritmo más adecuado de acuerdo con las características del grafo y del problema que se desea resolver.

2. Conceptos básicos de grafos

En el ámbito de las **matemáticas** y la **informática**, los **grafos** son una herramienta fundamental para ilustrar **relaciones entre objetos**. Aunque pueden parecer conceptos abstractos a primera vista, cuentan con diversas aplicaciones prácticas. Desde la **planificación de rutas** hasta las **redes sociales**, los **grafos** desempeñan un papel importante en muchas áreas de nuestra vida diaria.



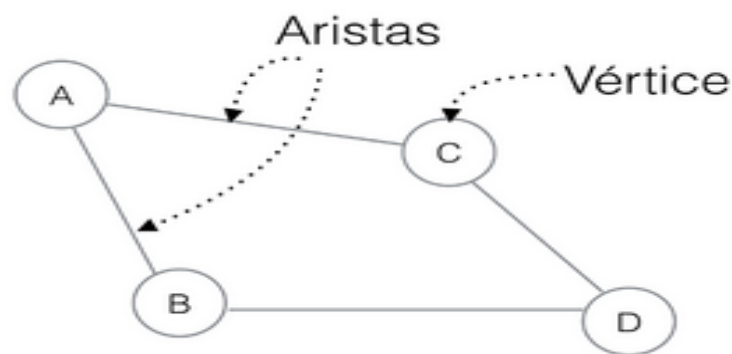
2.1. Definición de Grafos

Un grafo se define como un conjunto de puntos, denominados nodos o vértices, que están interconectados por líneas conocidas como aristas. Se representa formalmente de la siguiente manera: $G = (\text{Nodos}, \text{Aristas})$.

2.2. Vértices y aristas

Vértices ($\setminus(V)$): Son los puntos o unidades fundamentales del grafo. Representan entidades individuales como personas, ciudades o computadoras.

Aristas ($\setminus(E)$): Son las líneas o conexiones que unen un par de vértices. Representan la relación o el camino existente entre dos entidades.



2.3. Grafos dirigidos y no dirigidos

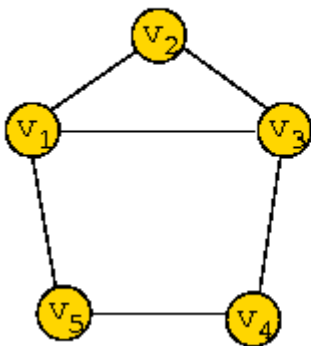
Grafos dirigidos

es un grafo donde las conexiones tienen una dirección definida. Esto significa que una conexión puede ir de un vértice a otro, pero no necesariamente en sentido contrario. Se representa con flechas.

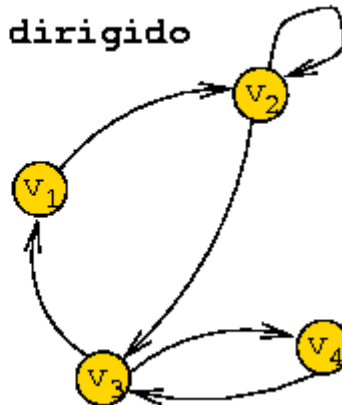
Grafos no dirigidos

es un grafo donde las conexiones no tienen una dirección específica. Si dos vértices están conectados, se puede recorrer la conexión en ambos sentidos.

Grafo no dirigido



Grafo dirigido

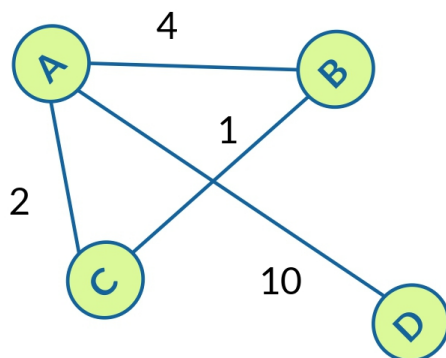


2.4. Grafos ponderados

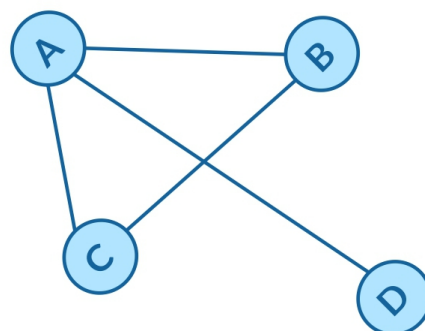
Un grafo ponderado es un grafo en el que cada conexión entre dos vértices tiene un valor o peso. Este valor puede representar una distancia, un costo, un tiempo u otra cantidad.

Estos grafos son importantes para encontrar el camino de menor costo entre dos puntos.

Ponderados



No Ponderados



2.5. Peso de una arista

El peso de una arista es el valor que se asigna a una conexión entre dos vértices. Este valor indica cuánto cuesta recorrer esa conexión.

2.6. Caminos, recorridos y ciclos

Recorrido: Es una secuencia de vértices en la que cada par de vértices consecutivos está conectado mediante una arista. Permite pasar de un vértice a otro siguiendo las conexiones del grafo.

Camino simple: Es un recorrido en el que no se repite ningún vértice. Esto permite representar una ruta sin pasar nuevamente por un punto que ya fue recorrido.

Ciclo : Es un camino que comienza y termina en el mismo vértice, sin repetir los demás vértices durante el recorrido. Por ejemplo: $A \rightarrow B \rightarrow C \rightarrow A$.



2.7. Costo de un camino

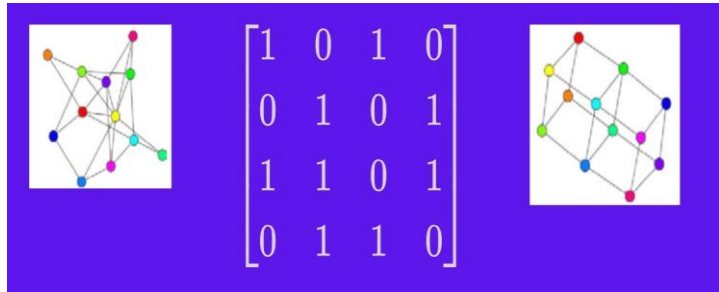
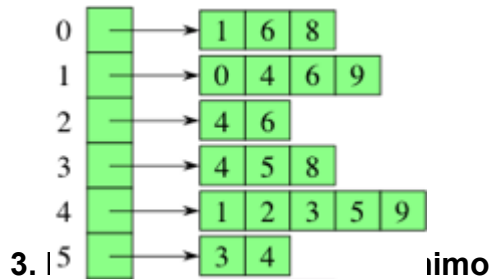
El costo de un camino es la suma de los pesos de todas las aristas que se recorren para llegar desde un vértice hasta otro. Por ejemplo, si un camino tiene pesos 3, 5 y 2, su costo total será:

$$3 + 5 + 2 = 10$$

2.8. Representación de grafos

La representación de un grafo es la forma en que se organizan sus vértices y aristas para facilitar su almacenamiento y análisis. Las formas más utilizadas son la **matriz de adyacencia** y la **lista de adyacencia**.

- **Matriz de adyacencia:** utiliza una tabla donde las filas y columnas representan los vértices. Cada posición indica si existe una conexión entre ellos y, en un grafo ponderado, puede almacenar el peso de la arista.
- **Lista de adyacencia:** para cada vértice se registra una lista con los vértices que están conectados a él y, si corresponde, el peso de cada conexión.

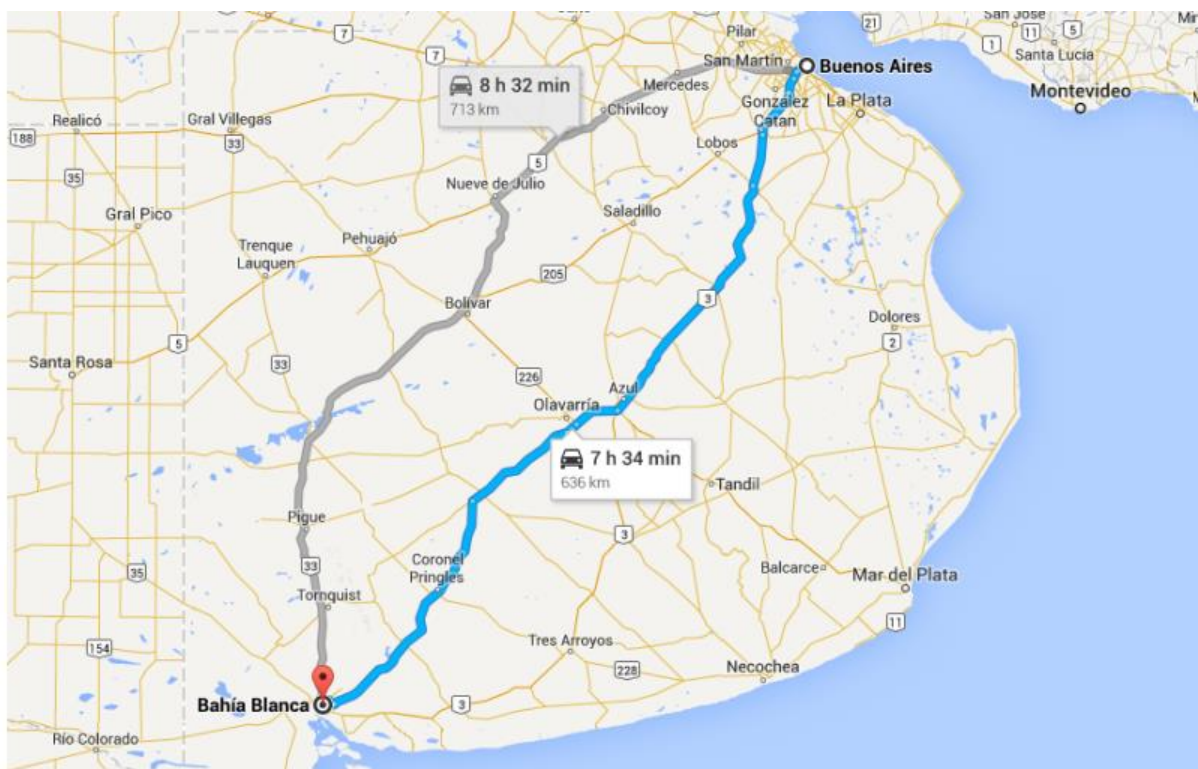


En esta parte del informe tratará sobre uno de los problemas más renombrados de la computación: el problema del camino mínimo.

3.1. Definición del problema del camino mínimo

El problema del camino mínimo consiste en encontrar la mejor forma de ir desde un punto de origen hasta otro punto de destino, buscando minimizar el costo del recorrido. Este costo puede representar la distancia recorrida, el tiempo empleado o cualquier otro valor asociado a las conexiones del grafo.

Cuando existen varias rutas posibles entre los puntos, se comparan sus costos y se selecciona aquella que presenta el menor valor. De esta manera, el problema del camino mínimo permite determinar una ruta óptima dentro de un grafo.

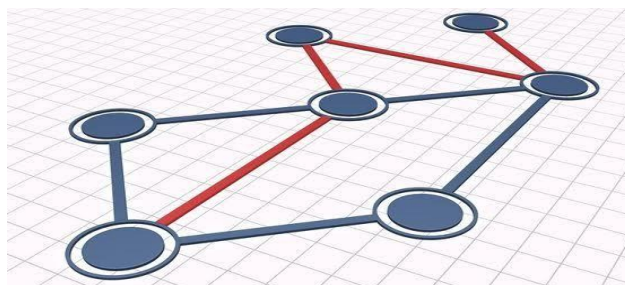


3.2. Origen y destino de un camino

En un camino dentro de un grafo se pueden identificar un vértice de origen y un vértice de destino. El vértice de origen es el punto desde donde comienza el recorrido, mientras que el vértice de destino es el punto al que se desea llegar.

Para determinar una ruta entre ambos, se siguen las aristas que conectan los diferentes vértices del grafo. Si existen varias rutas posibles, estas pueden compararse según su costo para determinar cuál es la más conveniente. En el problema del camino mínimo, se busca la ruta que permita llegar desde el origen hasta el destino con el menor costo posible.

Ejemplo: Si se desea encontrar una ruta desde el vértice A hasta el vértice D, entonces A representa el origen y D representa el destino. Una posible ruta sería $A \rightarrow B \rightarrow C \rightarrow D$. Si existen otras rutas, se comparan sus costos para determinar cuál es la de menor costo.



3.3. Distancia o costo mínimo

La distancia o el costo de un camino se determina al sumar los pesos de todas las aristas que componen la ruta. Estos pesos pueden representar distintos valores, como distancia, tiempo o costo, dependiendo del problema que se esté evaluando.

Cuando hay varias rutas entre un vértice de origen y uno de destino, se calcula el costo total de cada una y se comparan los resultados. El camino mínimo será aquel

que tenga la suma de pesos más baja. Por ejemplo, si una ruta tiene pesos 3, 5 y 2, su costo total es:

$$3 + 5 + 2 = 10$$

3.4. Caminos mínimos desde un origen

Este problema consiste en encontrar la distancia o el costo mínimo desde un vértice de origen hacia los demás vértices de un grafo. Es decir, en lugar de buscar una sola ruta, se busca conocer cuál es la mejor ruta desde un mismo punto hacia los diferentes puntos del grafo. Este problema es conocido como Single-Source Shortest Paths (SSSP).

Por ejemplo, si el vértice A es el origen, se busca determinar el menor costo para llegar desde A hasta B, C, D y los demás vértices. Para esto se pueden utilizar algoritmos como Dijkstra o Bellman-Ford, dependiendo de los pesos de las aristas.

3.5. Caminos mínimos entre todos los pares de vértices

Este problema consiste en encontrar el camino de menor costo entre todos los pares de vértices de un grafo. Esto significa que no se elige un solo punto de partida, sino que se analiza la mejor ruta posible entre los diferentes puntos del grafo.

Por ejemplo, si tenemos los vértices A, B, C y D, se busca conocer cuál es el menor costo para ir de A a B, de A a C, de A a D, y también entre los demás vértices. Para resolver este tipo de problema se puede utilizar el algoritmo Floyd-Warshall.

3.6. Pesos positivos y negativos

Los pesos de las aristas representan el costo de pasar de un vértice a otro. Estos valores pueden ser positivos o, en algunos problemas, negativos. Por ejemplo, una arista con peso 5 aumenta el costo del recorrido, mientras que una con peso -2 lo disminuye.

Esto es importante porque no todos los algoritmos funcionan de la misma manera con estos pesos. Dijkstra se utiliza cuando los pesos son no negativos, mientras que Bellman-Ford puede trabajar con pesos negativos. Por ello, conocer los pesos del grafo ayuda a elegir el algoritmo adecuado.

3.7. Ciclos negativos

Un ciclo negativo es un recorrido que comienza y termina en el mismo vértice y cuya suma de pesos es menor que cero. Por ejemplo, si un ciclo tiene pesos 2, -5 y 1, su suma es:

$$2 + (-5) + 1 = -2$$

El problema aparece porque, si este ciclo puede recorrerse repetidamente, el costo puede seguir disminuyendo. Por esta razón, no se puede establecer un costo mínimo normal para ese recorrido. El algoritmo Bellman-Ford puede detectar este tipo de ciclos.

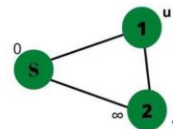
3.8. Importancia del problema del camino mínimo

El problema del camino mínimo es importante porque permite encontrar rutas que tengan el menor costo posible. El costo no necesariamente representa dinero; también puede representar distancia, tiempo o algún otro valor.

Por ejemplo, en un sistema de transporte se puede buscar la ruta más corta entre dos lugares, mientras que en una red de comunicación se puede buscar un recorrido que reduzca el tiempo o el costo de transmisión. Por ello, los algoritmos de caminos mínimos tienen aplicaciones en diferentes problemas relacionados con redes y recorridos.

4. Relajación de aristas

4.1. Definición de la relajación de una arista



La relajación de una arista es el proceso de comprobar si una conexión entre dos vértices permite encontrar un camino más corto. Para ello, se compara la distancia que ya se conoce hasta un vértice con una nueva distancia obtenida al pasar por otra arista.

Esta operación es importante porque permite ir mejorando las distancias conocidas hasta encontrar los caminos mínimos.

4.2. Funcionamiento de la relajación

Para realizar una relajación se compara la distancia actual de un vértice con la distancia que se obtendría pasando por otro vértice.

De forma sencilla:

Nueva distancia = distancia conocida + peso de la arista

Si la nueva distancia es menor que la que ya se tenía, se realiza una actualización. Si es mayor o igual, se mantiene la distancia anterior.

4.3. Actualización de distancias

Cuando se encuentra una ruta más corta, se reemplaza la distancia anterior por la nueva distancia. También puede registrarse el vértice anterior, llamado predecesor, para poder reconstruir posteriormente el camino encontrado.

Por ejemplo, si actualmente sabemos que llegar a B cuesta 10, pero encontramos una nueva ruta que cuesta 7, la distancia de B se actualiza de 10 a 7.

4.4. Ejemplo de relajación de una arista

Supongamos que tenemos:

$$A \rightarrow B = 5$$

y conocemos que la distancia desde el origen hasta A es 3. Entonces, pasando por A, podemos llegar a B con un costo de:

$$3 + 5 = 8$$

Si anteriormente se conocía una distancia de 10 para llegar a B, comparamos:

$$8 < 10$$

Como 8 es menor que 10, se actualiza la distancia de B a 8.

En cambio, si la nueva ruta tuviera un costo de 12, no se realizaría ningún cambio porque $12 > 10$. Este proceso de comparación y actualización es la idea principal de la relajación de aristas.

5. Algoritmo de Dijkstra

5.1. Definición y Principio del Enfoque Devorador (Greedy)

El algoritmo de Dijkstra, formulado por Edsger W. Dijkstra en 1956, resuelve el problema del camino mínimo desde un único origen (Single-Source Shortest Path - SSSP) en grafos dirigidos o no dirigidos con pesos en las aristas estrictamente no negativos ($w(u,v) \geq 0$).

El algoritmo opera bajo la estrategia devoradora (greedy): mantiene un conjunto S de vértices cuyos caminos mínimos desde la fuente s ya han sido determinados. En cada iteración, selecciona el vértice $u \notin S$ que posee la menor estimación de distancia provisional $d[u]$, lo añade al conjunto S y realiza el proceso de relajación sobre todas las aristas salientes de u .

5.2. Formalización Matemática y Propiedades

Sea un grafo ponderado $G = (V, E, w)$ con $w: E \rightarrow \mathbb{R}_{\geq 0}$ y un nodo origen $s \in V$. Se definen:

- $d[v]$: Cota superior actual de la distancia mínima desde s hasta v .
- $\pi[v]$: Predecesor de v en el camino mínimo.

Invariante de Bucle: Al inicio de cada iteración del bucle principal, para cada vértice $u \in S$, se cumple que $d[u] = \delta(s, u)$, donde $\delta(s, u)$ representa el valor del camino óptimo de s a u .

Condición de Fallo con Pesos Negativos: Si existe una arista con peso $w(u, v) < 0$, la propiedad devoradora se quiebra, pues un vértice ya insertado en S podría reducir su costo posteriormente a través de este arco negativo, llevando a un resultado incorrecto.

5.3. Pseudocódigo

Algoritmo Dijkstra (G, w, s):

Para cada v en $V[G]$:

$d[v] \leftarrow \text{INFINITO}$

$\pi[v] \leftarrow \text{NULO}$

$d[s] \leftarrow 0$

$Q \leftarrow \text{CrearColaDePrioridadMinima}(V[G])$ // Basada en $d[v]$

$S \leftarrow$ conjunto vacío

Mientras Q no esté vacía:

$u \leftarrow \text{ExtraerMinimo}(Q)$

$S \leftarrow S \cup \{u\}$

Para cada v adyacente a u :

Si $d[v] > d[u] + w(u, v)$ entonces:

$d[v] \leftarrow d[u] + w(u, v)$

$\pi[v] \leftarrow u$

$\text{DecrementarClave}(Q, v, d[v])$

6. Algoritmo de Bellman-Ford

El algoritmo de Bellman-Ford (desarrollado por Richard Bellman y Lester Ford Jr.) resuelve el problema SSSP en grafos dirigidos generales. A diferencia de Dijkstra, permite aristas con pesos negativos ($w(u, v) \in \mathbb{R}$).

Su fundamentación matemática reside en la programación dinámica y la propiedad del subcamino óptimo. Dado que un camino simple en un grafo con $|V|$ vértices contiene como máximo $|V| - 1$ aristas, el algoritmo garantiza la convergencia relajando todas las aristas del grafo $|V| - 1$ veces.

6.2. Detección de Ciclos Negativos

Si el grafo contiene un ciclo cuyos pesos suman un valor negativo alcanzable desde s , el concepto de camino mínimo deja de estar bien definido (se puede iterar indefinidamente sobre el ciclo disminuyendo el costo a $-\infty$).

Bellman-Ford detecta esta condición ejecutando una V -ésima iteración de relajación:

Si $\exists (u, v) \in E$ tal que $d[v] > d[u] + w(u, v) \Rightarrow$ Existe un ciclo de costo negativo.

6.3. Pseudocódigo

Algoritmo BellmanFord(G, w, s):

// Paso 1: Inicialización

Para cada v en $V[G]$:

$d[v] \leftarrow \text{INFINITO}$

$\pi[v] \leftarrow \text{NULO}$

$d[s] \leftarrow 0$

// Paso 2: Relajación sistemática $|V| - 1$ veces

Para i desde 1 hasta $|V| - 1$:

 Para cada arista (u, v) en $E[G]$:

 Si $d[u] \neq \text{INFINITO}$ y $d[v] > d[u] + w(u, v)$ entonces:

$d[v] \leftarrow d[u] + w(u, v)$

$\pi[v] \leftarrow u$

// Paso 3: Verificación de ciclos negativos

Para cada arista (u, v) en $E[G]$:

 Si $d[u] \neq \text{INFINITO}$ y $d[v] > d[u] + w(u, v)$ entonces:

 Retornar FALSO ("Ciclo negativo detectado")

Retornar VERDADERO

7. Algoritmo de Floyd-Warshall

7.1. Definición y Problema de Todos los Pares de Caminos Mínimos (APSP)

El algoritmo de Floyd-Warshall resuelve el problema de los caminos mínimos entre **todos los pares de vértices** (*All-Pairs Shortest Paths* - APSP) en un grafo dirigido y ponderado. Permite pesos negativos en las aristas, pero no ciclos negativos.

7.2. Formulación Recurrencial de Programación Dinámica

Sea $d_{ij}^{(k)}$ el peso del camino mínimo desde el vértice i hasta el vértice j considerando únicamente como vértices intermedios los elementos del conjunto $\{1, 2, \dots, k\}$.

La relación de recurrencia está dada por:

$$d_{ij}^{(0)} = \begin{cases} 0 & \text{si } i = j \\ w(i, j) & \text{si } (i, j) \in E \\ \infty & \text{si } i \neq j \text{ y } (i, j) \notin E \end{cases}$$

$$d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}) \quad \text{para } k = 1, 2, \dots, |V|$$

Interpretación: En el paso k , el camino mínimo entre i y j o bien no pasa por k (se conserva $d_{ij}^{(k-1)}$), o bien pasa por k (se divide en los subcaminos $i \rightarrow k$ y $k \rightarrow j$).

7.3. Pseudocódigo

Algoritmo FloydWarshall(W, V):

$D^{(0)} \leftarrow W$ // Matriz de pesos inicial

$P^{(0)} \leftarrow$ Matriz de predecesores inicial

Para k desde 1 hasta $|V|$:

Para i desde 1 hasta $|V|$:

Para j desde 1 hasta $|V|$:

Si $D^{(k-1)}[i][j] > D^{(k-1)}[i][k] + D^{(k-1)}[k][j]$ entonces:

$D^{(k)}[i][j] \leftarrow D^{(k-1)}[i][k] + D^{(k-1)}[k][j]$

$P^{(k)}[i][j] \leftarrow P^{(k-1)}[k][j]$

Sino:

$D^{(k)}[i][j] \leftarrow D^{(k-1)}[i][j]$

$P^{(k)}[i][j] \leftarrow P^{(k-1)}[i][j]$

Retornar $D^{(|V|)}$

8. Comparación de los Algoritmos

A continuación, se resume el análisis comparativo formal considerando el tipo de problema, complejidades temporales/espacial y restricciones del grafo:

Parámetro / Criterio	Algoritmo de Dijkstra	Algoritmo de Bellman-Ford	Algoritmo de Floyd-Warshall
Tipo de Problema	Fuente única (SSSP)	Fuente única (SSSP)	Todos los pares (APSP)
Paradigma	Devorador (<i>Greedy</i>)	Programación Dinámica	Programación Dinámica
Complejidad Temporal (Arreglo)	$\mathcal{O}(V ^2)$	$\mathcal{O}(V \cdot E)$	$\mathcal{O}(V ^3)$
Complejidad Temporal (Heap Binario)	$\mathcal{O}((V + E) \log V)$	N/A	N/A
Complejidad Temporal (Heap Fibonacci)	$\mathcal{O}(E + V \log V)$	N/A	N/A
Complejidad Espacial	$\mathcal{O}(V)$	$\mathcal{O}(V)$	$\mathcal{O}(V ^2)$
Pesos Negativos	No permitidos ($w \geq 0$)	Permitidos	Permitidos
Detección de Ciclos Negativos	No	Sí (Devuelve FALSO)	Sí (Valores negativos en la diagonal $D[i][i] < 0$)

9. Selección del Algoritmo Adecuado

Para seleccionar la estrategia óptima en la resolución de problemas reales o teóricos, se debe seguir la siguiente matriz de decisión:

1. Si se busca el camino mínimo desde una sola fuente (SSSP):

a. ¿Todas las aristas tienen pesos no negativos ($w \geq 0$)?

- SÍ:** Utilizar **Dijkstra**. Con implementación mediante *Fibonacci Heap* u *Orden de Prioridad Binario*, ofrece el rendimiento computacional más eficiente: $\mathcal{O}(|E| + |V| \log |V|)$.
- NO:** Utilizar **Bellman-Ford**. Maneja aristas negativas con una complejidad $\mathcal{O}(|V| \cdot |E|)$ y verifica si el problema es resoluble (detectando ciclos negativos).

2. Si se busca el camino mínimo entre todos los pares de nodos (APSP):

- Para Grafos Densos ($|E| \approx |V|^2$):** Utilizar **Floyd-Warshall** ($\mathcal{O}(|V|^3)$). Su estructura basada en matrices simples posee un factor constante muy bajo e implementación altamente optimizable en caché.
- Para Grafos Dispersos ($|E| \ll |V|^2$) con pesos no negativos:** Ejecutar **Dijkstra** $|V|$ veces, obteniendo una complejidad de $\mathcal{O}(|V| \cdot |E| \log |V|)$, la cual es superior a $\mathcal{O}(|V|^3)$.
- Para Grafos Dispersos con pesos negativos:** Utilizar el **Algoritmo de Johnson** (que combina Bellman-Ford para reponderar el grafo y

Dijkstra $|V|$ veces), logrando una complejidad de $\mathcal{O}(|V|^2 \log |V| + |V||E|)$

10. Ejemplo Práctico Comparativo

10.1. Definición del Grafo de Prueba

Consideremos el siguiente grafo dirigido $G = (V, E)$ con $V = \{A, B, C, D\}$ y la matriz de pesos W :

- $w(A, B) = 2$
- $w(A, C) = 5$
- $w(B, C) = -4$ (Nota: Arista de peso negativo)
- $w(B, D) = 1$
- $w(C, D) = 1$

Tomaremos como **origen el nodo A**.

10.2. Ejecución con Algoritmo de Dijkstra (Fallo demostrativo)

1. **Inicialización:** $d[A] = 0, d[B] = \infty, d[C] = \infty, d[D] = \infty, S = \emptyset$.
2. **Paso 1:** Extraer A ($d[A] = 0$). $S = \{A\}$.
 - a. Relajar (A, B) : $d[B] = \min(\infty, 0 + 2) = 2$.
 - b. Relajar (A, C) : $d[C] = \min(\infty, 0 + 5) = 5$.
3. **Paso 2:** Vértice fuera de S con menor distancia es B ($d[B] = 2$). Extraer B . $S = \{A, B\}$.
 - a. Relajar (B, C) : $d[C] = \min(5, 2 + (-4)) = -2$.
 - b. Relajar (B, D) : $d[D] = \min(\infty, 2 + 1) = 3$.
4. **Paso 3:** Vértice fuera de S con menor distancia es C ($d[C] = -2$). Extraer C . $S = \{A, B, C\}$.
 - a. Relajar (C, D) : $d[D] = \min(3, -2 + 1) = -1$.
5. **Resultado Dijkstra:** $d[A] = 0, d[B] = 2, d[C] = -2, d[D] = -1$.
 - *Nota metodológica:* En este grafo en particular, Dijkstra ajustó la distancia de C porque la arista negativa salía de un nodo no procesado antes que C . Sin embargo, si la arista (B, C) se procesara después de cerrar C , Dijkstra fallaría al no reevaluar nodos ya ingresados en S . Por ello, no se garantiza corrección teórica formal en Dijkstra con aristas negativas.

10.3. Ejecución con Algoritmo de Bellman-Ford (Correcto)

1. **Inicialización:** $d[A] = 0, d[B] = \infty, d[C] = \infty, d[D] = \infty$.
2. **Iteración 1** ($i = 1$):
 - a. Arista (A, B) : $d[B] = 0 + 2 = 2$.

- b. Arista $(A, C): d[C] = 0 + 5 = 5$.
 - c. Arista $(B, C): d[C] = \min(5, 2 + (-4)) = -2$.
 - d. Arista $(B, D): d[D] = \min(\infty, 2 + 1) = 3$.
 - e. Arista $(C, D): d[D] = \min(3, -2 + 1) = -1$.
3. **Iteraciones 2 y 3** ($i = 2, 3$): No ocurren cambios en el arreglo d .
4. **Verificación de ciclos negativos:**
- a. Para toda arista no se cumple $d[v] > d[u] + w(u, v)$. No hay ciclos negativos.
5. **Resultado Bellman-Ford:**
- a. $d[A] = 0$
 - b. $d[B] = 2$ (Camino: $A \rightarrow B$)
 - c. $d[C] = -2$ (Camino: $A \rightarrow B \rightarrow C$)
 - d. $d[D] = -1$ (Camino: $A \rightarrow B \rightarrow C \rightarrow D$)

10.4. Ejecución con Algoritmo de Floyd-Warshall (Todos los Pares)

Matriz inicial $D^{(0)}$:

$$D^0 = \begin{pmatrix} 0 & 2 & 5 & \infty \\ \infty & 0 & -4 & 1 \\ \infty & \infty & 0 & 1 \\ \infty & \infty & \infty & 0 \end{pmatrix}$$

- **Paso $k = 1$ (Intermedio A):** Sin cambios aplicables.
- **Paso $k = 2$ (Intermedio B):**
 - $D_{AC}^{(2)} = \min(D_{AC}^{(1)}, D_{AB}^{(1)} + D_{BC}^{(1)}) = \min(5, 2 + (-4)) = -2$
 - $D_{AD}^{(2)} = \min(D_{AD}^{(1)}, D_{AB}^{(1)} + D_{BD}^{(1)}) = \min(\infty, 2 + 1) = 3$
- **Paso $k = 3$ (Intermedio C):**
 - $D_{AD}^{(3)} = \min(D_{AD}^{(2)}, D_{AC}^{(2)} + D_{CD}^{(2)}) = \min(3, -2 + 1) = -1$
- **Paso $k = 4$ (Intermedio D):** Sin cambios.

Matriz Final $D^{(4)}$:

$$D^4 = \begin{pmatrix} 0 & 2 & -2 & -1 \\ \infty & 0 & -4 & -3 \\ \infty & \infty & 0 & 1 \\ \infty & \infty & \infty & 0 \end{pmatrix}$$

(La primera fila de $D^{(4)}$ coincide con los resultados SSSP obtenidos desde la fuente A).

11. Aplicaciones reales

- **Sistemas de navegación y GPS:** Aplicaciones como Google Maps o Waze utilizan variantes de los algoritmos de Dijkstra y A* para calcular la ruta más rápida entre ubicaciones, integrando variables dinámicas como el tráfico en tiempo real.
- **Enrutamiento en redes de telecomunicaciones:** Protocolos de enrutamiento como OSPF (Open Shortest Path First) emplean el algoritmo de Dijkstra para dirigir paquetes de datos a través de la ruta más eficiente entre servidores y *routers*.
- **Logística y cadena de suministro:** Optimización de trayectos para flotas de entrega y transporte de mercancías, minimizando el consumo de combustible y los tiempos de distribución.
- **Análisis de redes sociales:** Cálculo de los grados de separación entre individuos y optimización de sugerencias de conexión mediante la identificación de los caminos más cortos en grafos de relaciones.

12. Implementación computacional

La eficiencia en la implementación de estos algoritmos depende críticamente de las estructuras de datos utilizadas:

- **Representación del grafo:** Se recomienda el uso de **listas de adyacencia** para grafos dispersos (comunes en mapas y redes) para minimizar el uso de memoria. Las **matrices de adyacencia** se reservan para grafos muy densos.
- **Colas de prioridad:** Para algoritmos avaros como Dijkstra o A*, es indispensable implementar montículos binarios (*Min-Heaps*) o montículos de Fibonacci para extraer el vértice con el costo mínimo de forma rápida.

- **Entornos de ejecución:** Debido a la alta carga computacional en grafos masivos, estas implementaciones suelen desarrollarse en lenguajes de alto rendimiento y bajo nivel de gestión de memoria, como C++, Rust o Java.

13. Complejidad computacional

El costo computacional varía según el algoritmo, el número de vértices (V) y el número de aristas (E):

- **Dijkstra:** Con una cola de prioridad basada en montículos de Fibonacci, su complejidad es $O(E + V \log V)$. Es sumamente eficiente, pero no soporta pesos negativos.
- **Bellman-Ford:** Su complejidad temporal es $O(V \cdot E)$. Aunque es más lento que Dijkstra, es el estándar obligatorio cuando existen aristas con pesos negativos y permite detectar ciclos negativos.
- **Floyd-Warshall:** Calcula los caminos mínimos entre todos los pares de nodos simultáneamente. Su complejidad es $O(V^3)$, limitando su uso a grafos pequeños.
- **A*:** Su complejidad temporal teórica en el peor de los casos coincide con Dijkstra, pero en la práctica se reduce significativamente al podar el espacio de búsqueda gracias a su función heurística.

14. Conclusiones

El cálculo de caminos mínimos constituye un pilar esencial en la algoritmia moderna y la optimización de recursos. La principal conclusión es que no existe un "algoritmo definitivo" para todos los escenarios. La selección depende estrictamente de las restricciones matemáticas del problema (presencia de pesos negativos, necesidad de rutas únicas vs. todos los pares) y de la escala de la red abordada. Estos algoritmos han demostrado ser escalables y son la base operativa de infraestructuras globales como internet y la logística mundial.

15. Recomendaciones

- **Perfilado de datos:** Analizar la topología del grafo (densidad, tipo de pesos) antes de escribir código. Utilizar Bellman-Ford solo si está garantizada la posibilidad de pesos financieros o físicos negativos.
- **Uso de heurísticas:** En aplicaciones interactivas (videojuegos, navegación vehicular), priorizar siempre el algoritmo A* con heurísticas admisibles (como la distancia euclidiana o Manhattan) para garantizar respuestas en tiempo real sin sacrificar precisión.
- **Optimización de memoria:** Por defecto, estructurar los sistemas para utilizar listas de adyacencia y evitar las matrices de adyacencia a menos que la cantidad de nodos sea inferior a unos pocos miles, previniendo así desbordamientos de memoria (*Out of Memory*).

Bibliografía

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.

https://www.tutorialspoint.com/graph_theory/graph_theory_path_cycle.htm

[https://ninos.kiddle.co/Arista_\(teor%C3%ADa_de_grafos\)](https://ninos.kiddle.co/Arista_(teor%C3%ADa_de_grafos))

https://www.cs.swarthmore.edu/courses/CS35/S25/book/Chapter13.pdf?utm_source=chatgpt.com

https://users.ece.utexas.edu/~garg/publicCompanion/chapter7.html?utm_source=chatgpt.com

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press. (Capítulo 22: *Single-Source Shortest Paths* y Capítulo 23: *All-Pairs Shortest Paths*).

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271.

Bellman, R. (1958). On a routing problem. *Quarterly of Applied Mathematics*, 16(1), 87–90.

Floyd, R. W. (1962). Algorithm 97: Shortest path. *Communications of the ACM*, 5(6), 345.

Dasgupta, S., Papadimitriou, C. H., & Vazirani, U. V. (2006). *Algorithms*. McGraw-Hill Education.